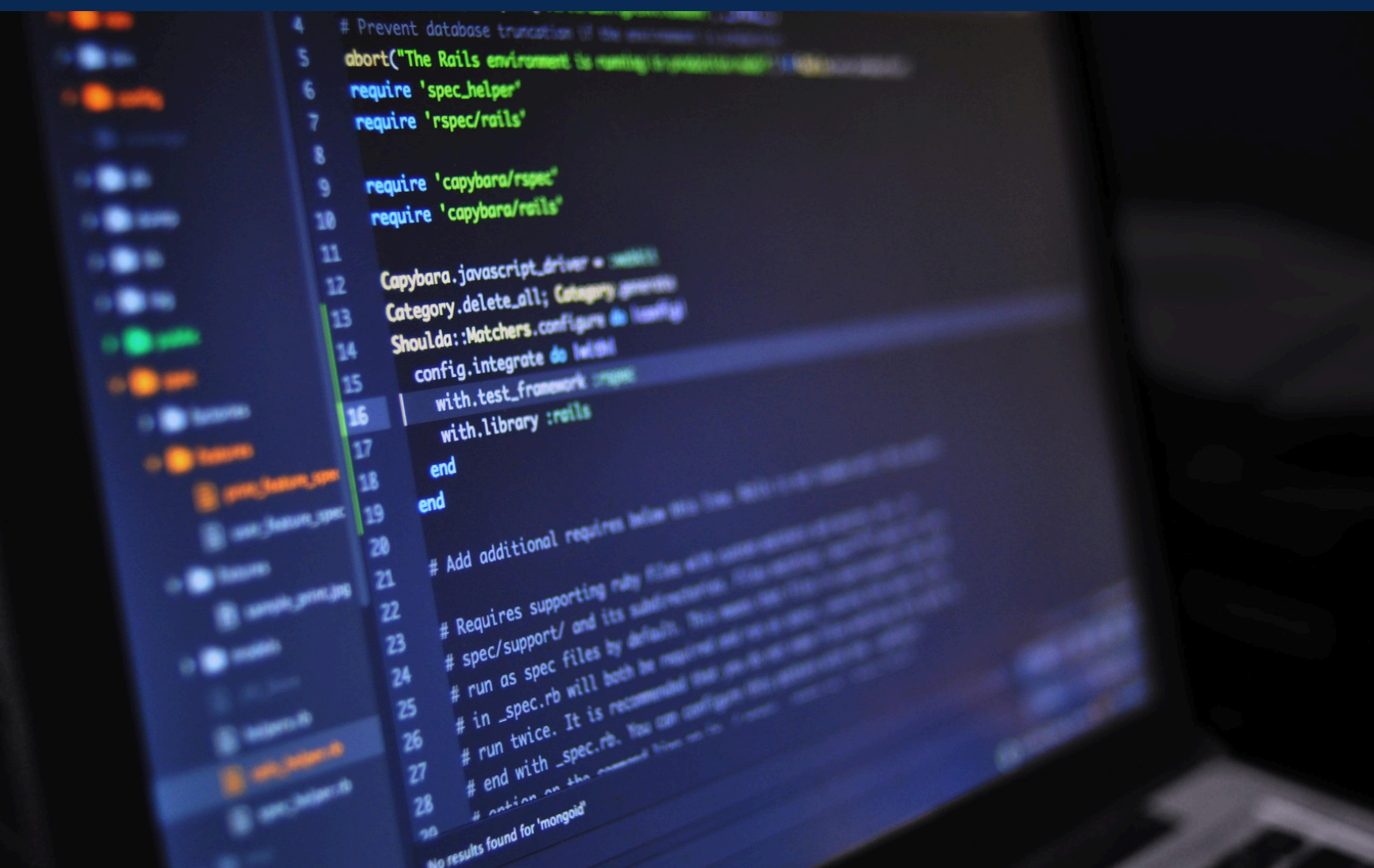


PROYECTO FINAL PROGRAMACIÓN

HECHO POR DENIS GANDRABUR
1º DAW

INDICE DE CONTENIDOS

1. INTRODUCCIÓN	4-8
2. CREACIÓN DE CLASES, HERENCIA	10
3. CONEXIÓN JDBC	12
4. DISEÑO DE LAS VENTANAS	14
5. UNIÓN DE LA GUI CON LA LÓGICA Y BDD	16
6. PRUEBAS DE ERRORES Y DOCUMENTACIÓN	18-19



1. INTRODUCCIÓN

TEMA

He elegido hacer un hotel porque lo veía cómo un proyecto muy completo que cumple todos los requisitos del proyecto. Le he puesto el nombre “Hotel La Mancha” porque suena muy elegante y el objetivo es de promover el turismo rural.

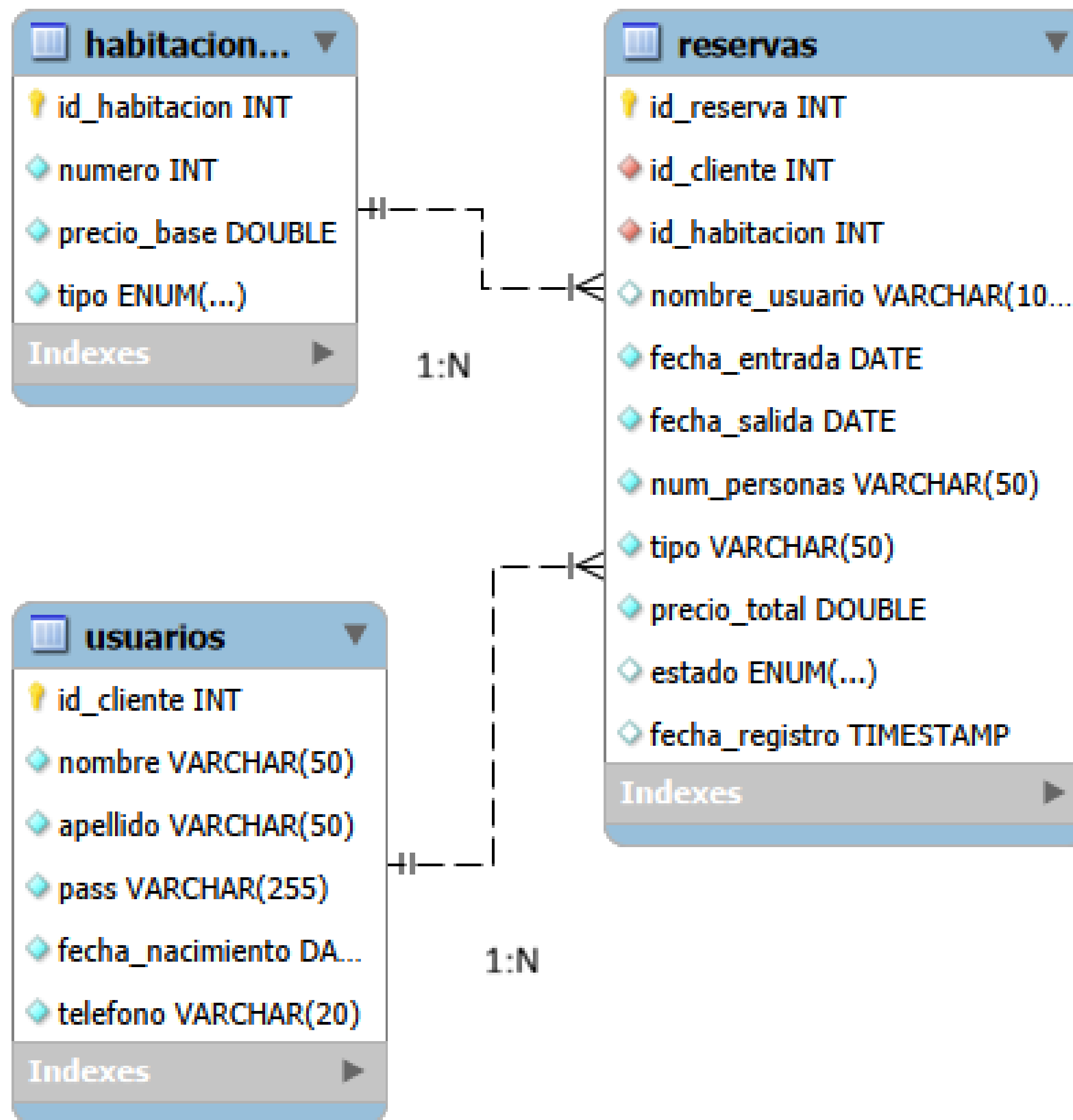
PROYECTO FINAL PROGRAMACIÓN



DISEÑO DEL DIAGRAMA UML



ESQUEMA BASE DE DATOS



2. CREACIÓN DE CLASES Y HERENCIA



Mi proyecto en total tiene en total 17 clases, incluyendo una herencia.

Las clases están organizadas por diferentes paquetes.

ProyectoFinal.BaseDeDatos incluye las clases: UsuarioDAO.java, HabitaciónDAO.java y ConexionBBDD.java.

ProyectoFinal.Configuración: JConfiguración.java, GestorConfiguracion.java y Configuracion.java.

ProyectoFinal.GUI: CrearCuenta.java, IniciarSesión.java, VentanaGaleria.java, TablaReservas.java, PantallaPrincipal.java, Starter.java.

ProyectoFinal.Hotel: Habitacion.java, HabitacionSimple.java, HabitacionDoble.java y Suite.java.

CONEXIÓN JDBC Y MÉTODOS DE ACCESO A DATOS



Conexión centralizada: La clase ConexionBBDD.java mediante el driver JDBC se encarga de mantener la comunicación entre la aplicación y la base de datos.

Las clases DAO: UsuarioDAO y HabitacionDAO se encargan de separar la lógica del negocio y devolver las consultas SQL.

Métodos CRUD: la aplicación tiene varios métodos para hacer un Insert, Update, Delete y Select.

Seguridad y rendimiento: Uso de PreparedStatement que permite interactuar con la base de datos.

DISEÑO DE LAS
VENTANAS
PRINCIPALES Y
NAVEGACIÓN.

PARA CREAR LAS INTERFACES, UTILICÉ
BOTONES, LABELS, TABLAS, CUADROS
COMBINADO, DATACHOOSER(JAVASWING
NO LO TRAE).

Design Preview [CrearCuenta]



CREAR CUENTA

Nombre:

Apellido:

Contraseña:

Fecha De Nacimiento:

Teléfono:

VOLVER

CREAR CUENTA

Panel de Control de Reservas - Administrador

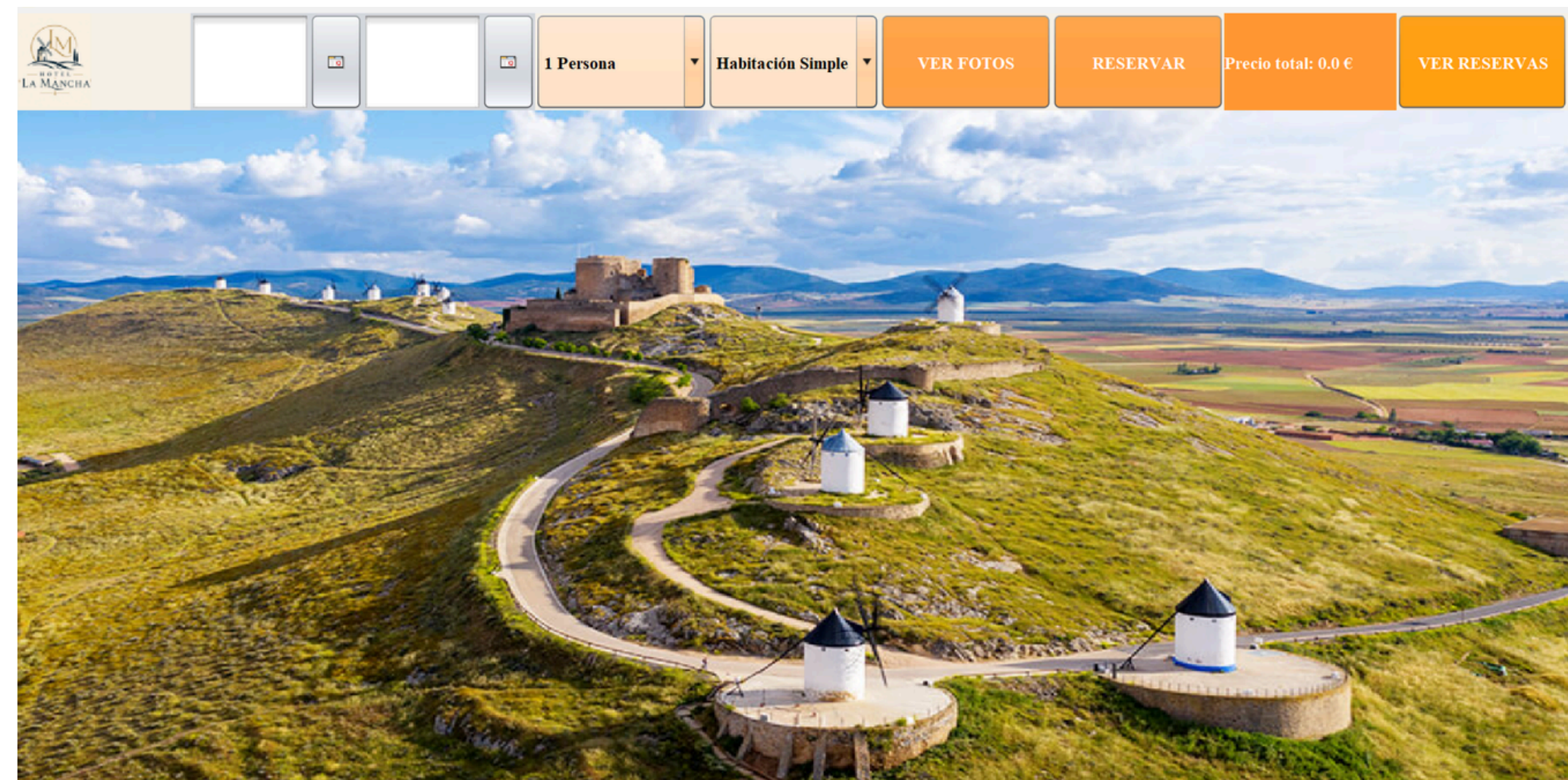


Modificar

Cancelar

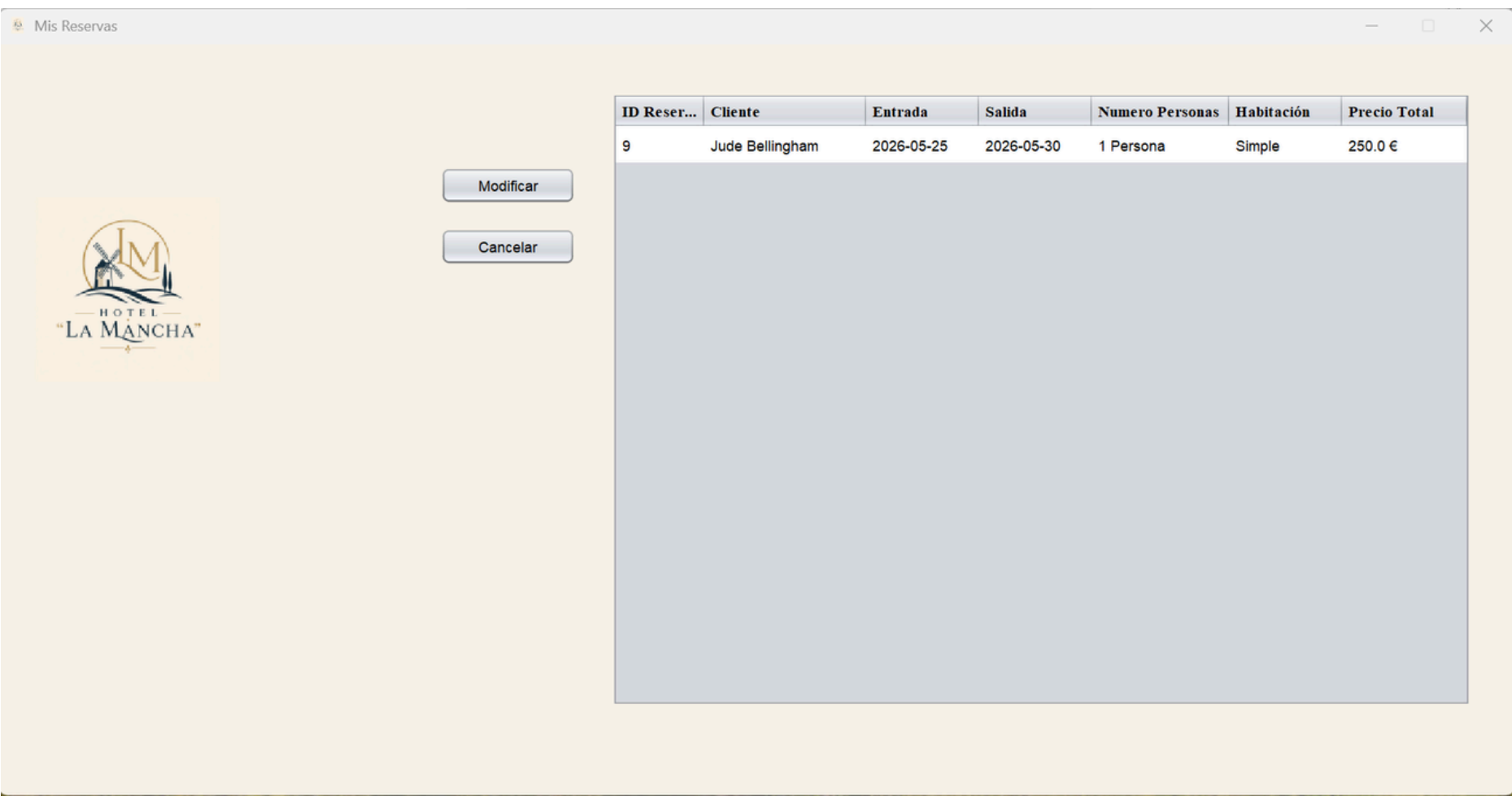
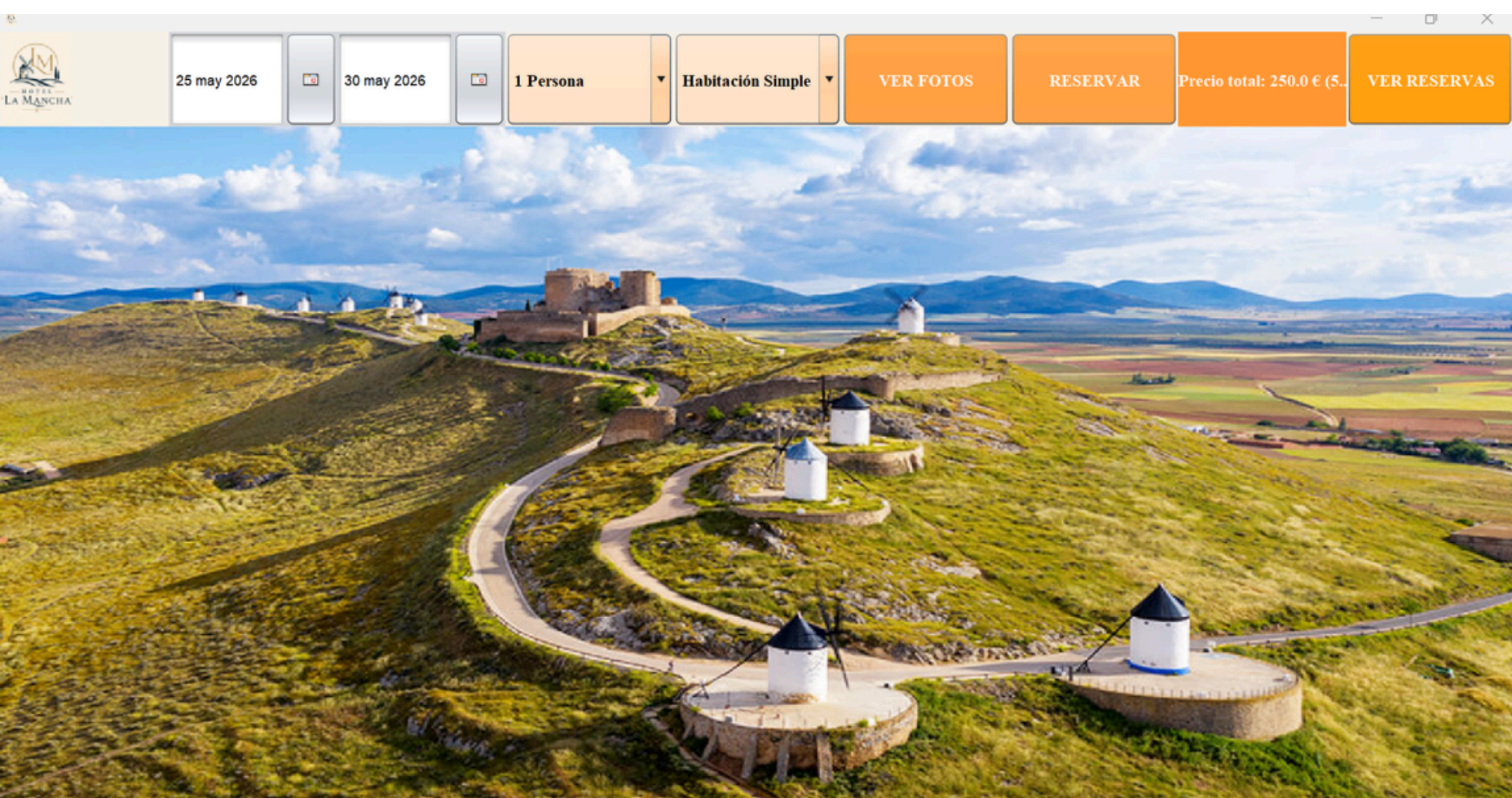
Eliminar

ID Reser...	Cliente	Entrada	Salida	Numero Personas	Habitación	Precio
5	Denis Gr	2026-05-24	2026-05-31	3 Personas	Suite	700.
8	Mark Pellegrino	2026-06-01	2026-06-30	2 Personas	Suite	2900



5.UNIÓN DE LA GUI CON LA LÓGICA Y LA BD.

- Uso de ActionEvents. Capturan las acciones del usuario. (Clic de botones, fechas).
- Gestión Transaccional. Ejecución de consultas en la base de datos que devuelven el estado de operación.
- Alertas del sistema. La aplicación utiliza JOptionPane para confirmar al cliente las acciones que ha realizado.
- Gestión Transaccional. Traspaso de información desde los formularios hasta los métodos DAO.

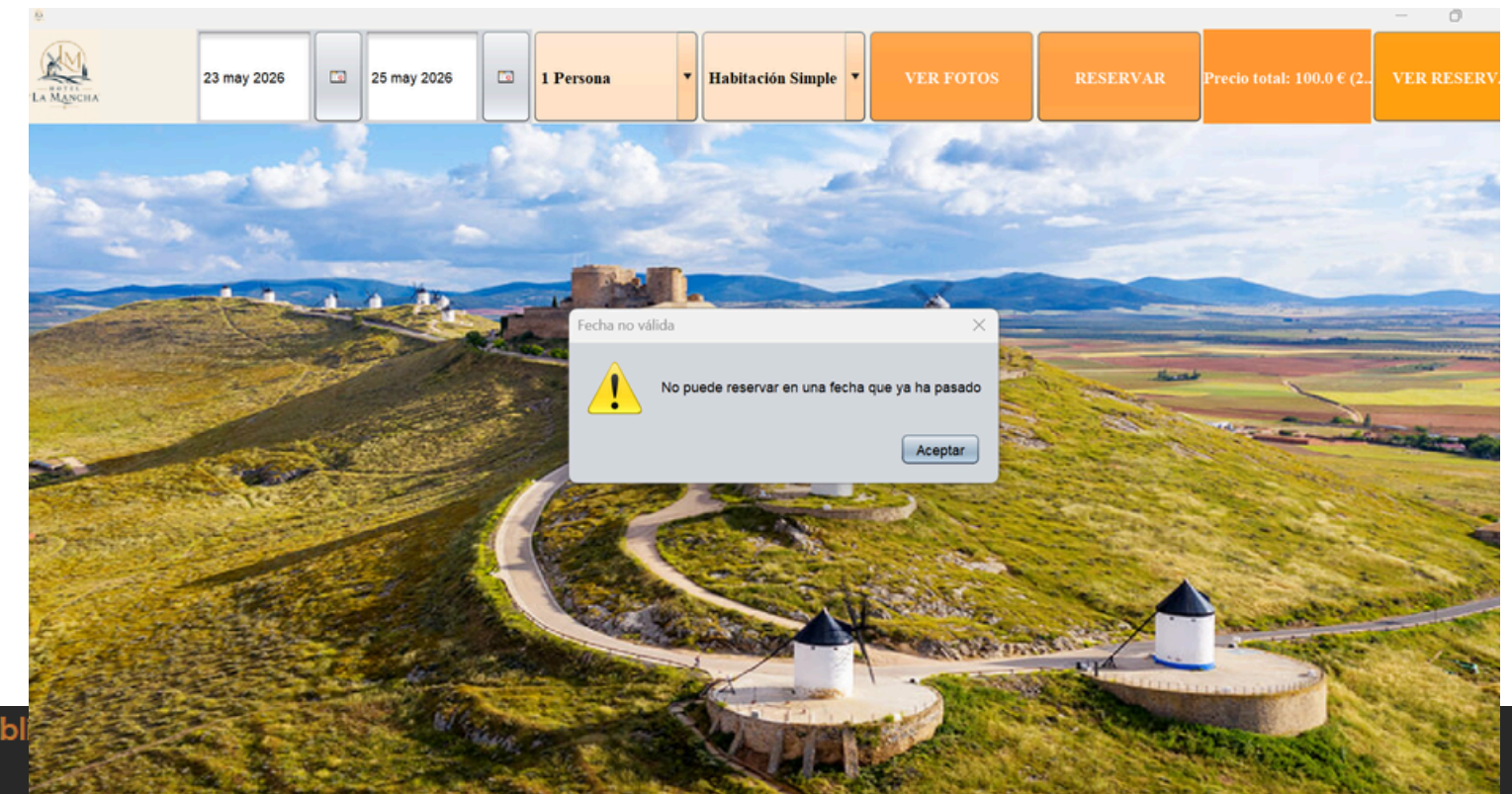


6.PRUEBAS DE
ERRORES,
DOCUMENTACIÓN
FINAL

Control de excepciones: Uso de bloques try - catch para prevenir caídas de red, archivos.

Errores entorno Cliente. Control de errores de lógica de negocio.

```
public static void main(String[] args) throws SQLException {
    Inicializar(null);
    try {
        Connection cn = getConexion();
        System.out.println("¡Conexión exitosa!:" + cn);
        cn.close();
    } catch (SQLException e) {
        System.out.println("Error de conexión: " + e.getMessage());
    }
}
```



```
public void cargarConfiguracion() {
    Configuracion confLeida = null;
    try {
        FileInputStream FIS = new FileInputStream(FICHERO_CONFIGURACION);
        ObjectInputStream OIS = new ObjectInputStream(FIS);
        confLeida = (Configuracion) OIS.readObject();
        OIS.close();
    } catch (FileNotFoundException e) {
        System.err.println("Error: Archivo de configuración no encontrado en " + FICHERO_CONFIGURACION);
        e.printStackTrace();
    } catch (IOException e) {
        System.err.println("Error de entrada/salida al leer la configuración");
        e.printStackTrace();
    } catch (ClassNotFoundException e) {
        System.err.println("Error: Clase Configuracion no encontrada");
        e.printStackTrace();
    }

    return confLeida;
}
```



Cómo conclusión final, el desarrollo de esta aplicación ha sido un reto que me ha servido para aprender y mejorar mis habilidades como programador. Aprendí a desarrollar una aplicación que pueda controlar los errores y que conecta las interfaces con la lógica. Ha sido un orgullo desarrollar esta aplicación y por fin puedo decir que “He creado algo”.

MUCHAS GRACIAS

```
    }  
    render() {  
      return (  
        <React.Fragment>  
          <div className="py-5">  
            <div className="container">  
              <Title name="our" title="product">  
                <div className="row">  
                  <ProductConsumer>  
                    {(value) => {  
                      console.log(value)  
                    }}  
                  </ProductConsumer>  
                </div>  
              </div>  
            </div>  
          </React.Fragment>  
        )  
      )  
    }  
  }  
}
```